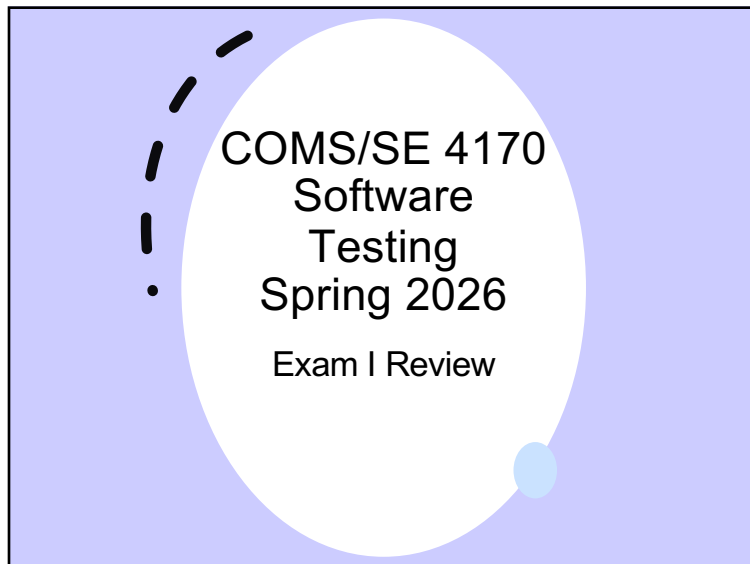
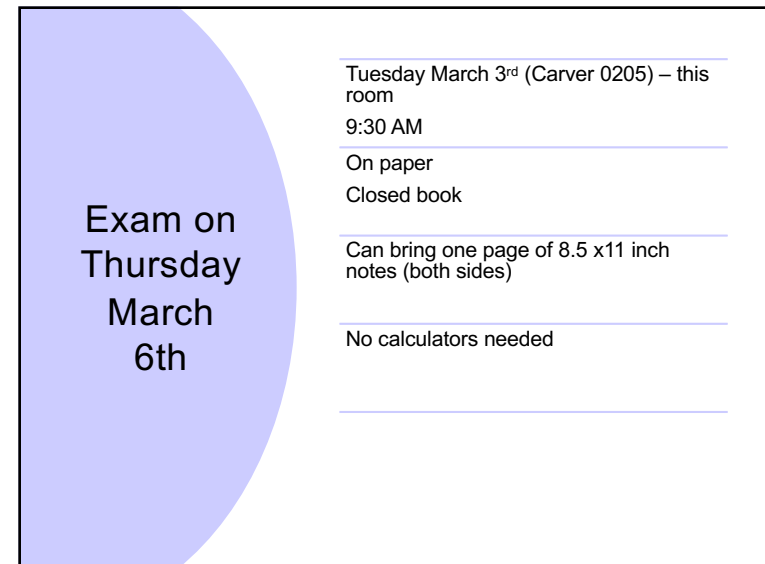
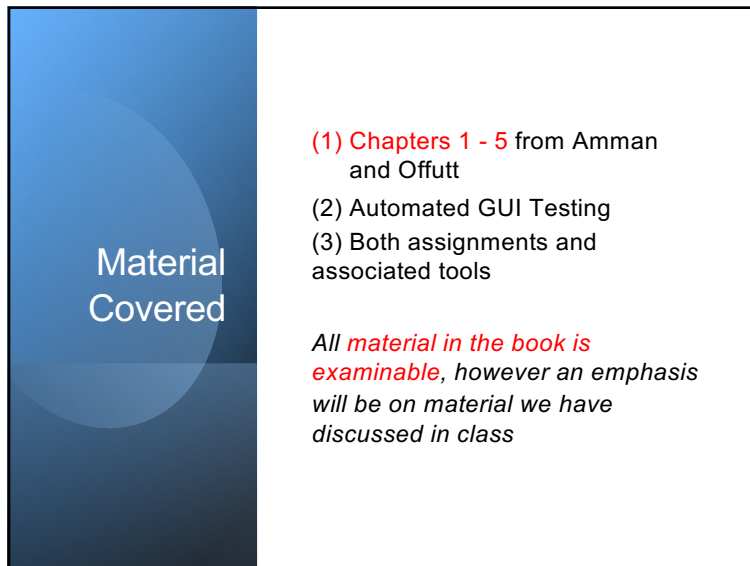




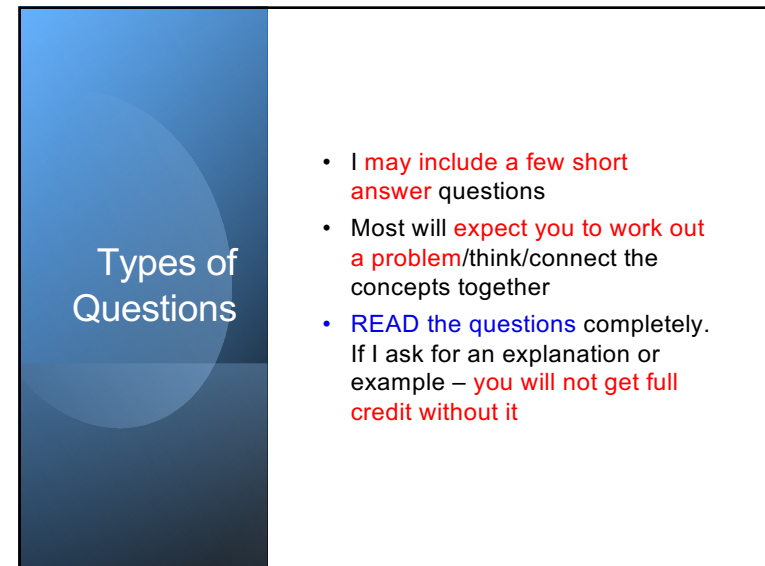
1



2



3



4

Topics

- Chapter 1: Overview of Software Testing
- Chapter 2: Model Driven test design
- Chapter 3: Test automation
- Chapter 4: Test driven development
- Automated GUI Testing
- Chapter 5: Test Coverage
- Assignments: jacoco code coverage, Evosuite
- Exercises: AFL

5

Overview of Testing

- Faults, failures and errors
- Levels of test maturity

6

A Concrete Example

```
public static int numZero (int [] arr)
{ // Effects: If arr is null throw NullPointerException
  // else return the number of occurrences of 0 in arr
  int count = 0;
  for (int i = 1; i < arr.length; i++)
  {
    if (arr [ i ] == 0)
    {
      count++;
    }
  }
  return count;
}
```

Fault: Should start searching at 0, not 1

Test 1
[2, 7, 0]
Expected: 1
Actual: 1

Test 2
[0, 2, 7]
Expected: 1
Actual: 0

Error: i is 1, not 0, on the first iteration
Failure: none

Error: i is 1, not 0
Error propagates to the variable count
Failure: count is 0 at the return statement

Introduction to Software Testing, Edition 2 (Ch 1)

© Ammann & Offutt

7

7

Testing Goals Based on Test Process Maturity

- **Level 0** : There's no difference between testing and debugging
- **Level 1** : The purpose of testing is to show correctness
- **Level 2** : The purpose of testing is to show that the software doesn't work
- **Level 3** : The purpose of testing is not to prove anything specific, but to reduce the risk of using the software
- **Level 4** : Testing is a mental discipline that helps all IT professionals develop higher quality software

Introduction to Software Testing, Edition 2 (Ch 1)

© Ammann & Offutt

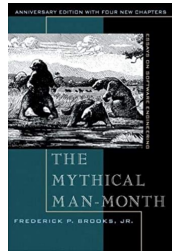
8

8

Key Challenge

*"Software is inherently complicated and our ultimate goal, **completely correct software is unreachable**"*

Accidental versus essential complexity



9

Key Challenge

*"Software is inherently complicated and our ultimate goal, **completely correct software is unreachable**"*

If we can't reach this goal, we need to be able to **observe that our software is acceptable** with respect to "**reliability, safety, maintainability, security, efficiency**"

10

Limitations

- Testing can only **show the presence of faults**. It **cannot** determine the absence of faults.



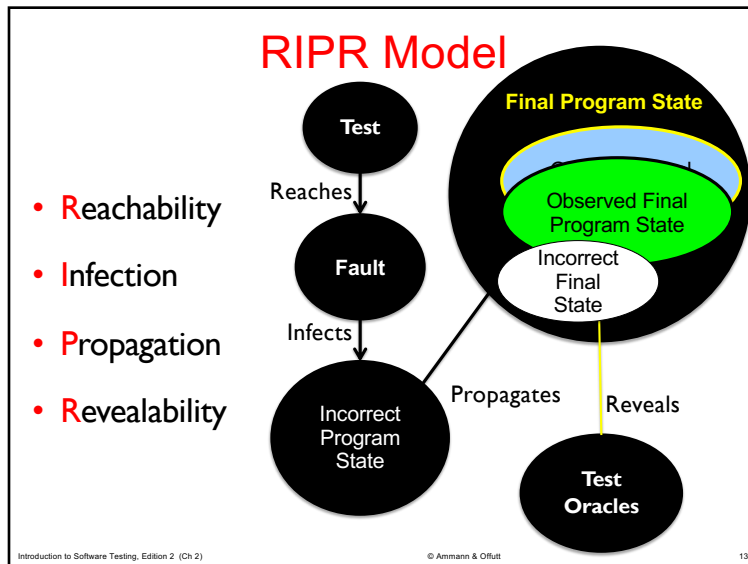
11

Model-Driven Test Design

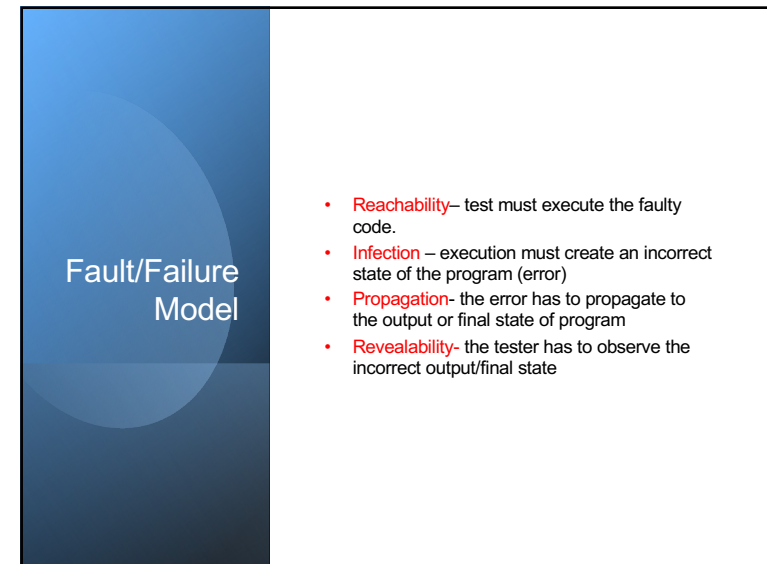
We can use higher levels of abstraction or **models** to reduce complexity and guide testing

- Helps with test case **design**
 - Use human knowledge and/or coverage
- Test case **generation**
- Test **sufficiency** (how much behavior/code have we covered)
- Test **prioritization** (which tests to run first)

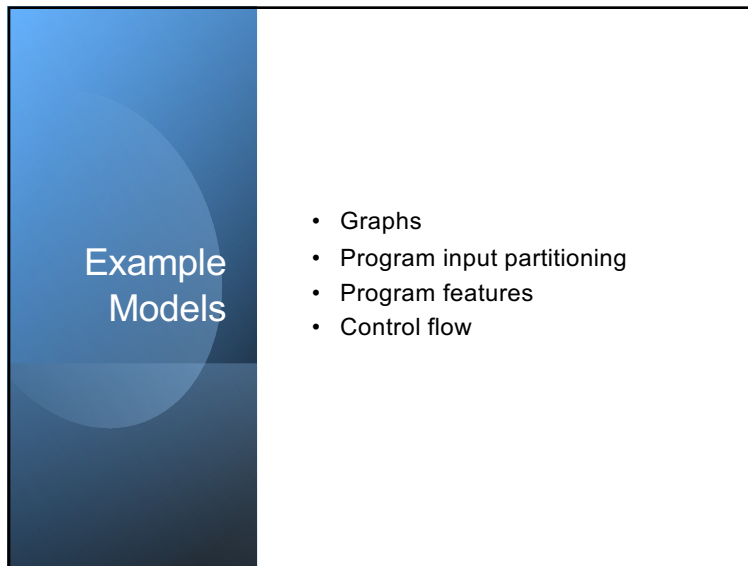
12



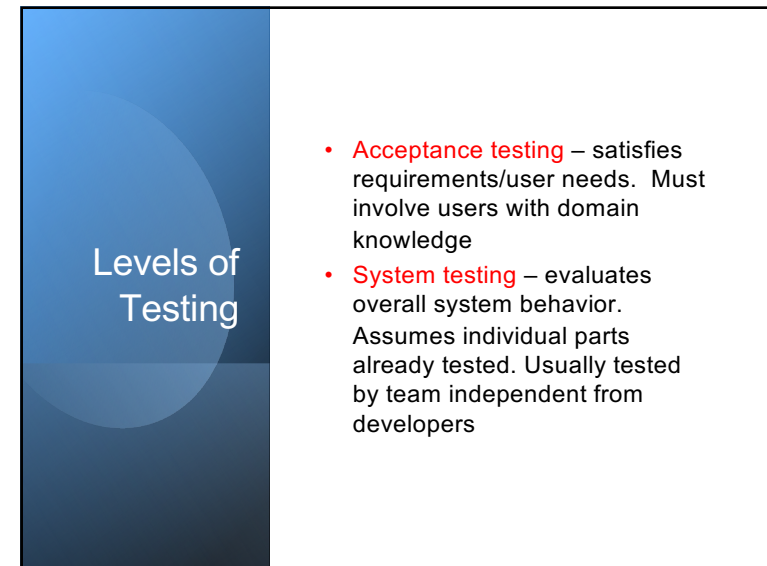
13



14



15



16

Developer testing levels

- **Integration testing** – evaluates subsystem design (puts modules together)
- **Module testing** – evaluates design of individual modules
- **Unit testing** – tests individual functions/units with respect to implementation

17

Regression Testing

- Performed throughout **program maintenance**
- **Each time a program is modified** (to fix a fault or add new functions)
- Ensures that the **previously working code has not broken** (regressed)
- Expensive because it happens often

18

Specification and Structural Testing

- **Specification**– derive tests from the specification or external description (no need to have code). Often used at system level
- **Structural**– derive tests from code. Often used at unit level
- However, both can be used at either level and can be combined

19

Model Driven Test Design

- Four parts:
 - Test design
 - Test automation
 - Test execution
 - Test evaluation

20

Test Automation

- Chapter 3

21

Benefits of Automation

- Reduces **cost** of testing
- Reduces **time of testing** – we can test more!
- Reduces **human error**
- Makes regression testing easier
 - can be executed by click of a button

22

Types of SE Tasks

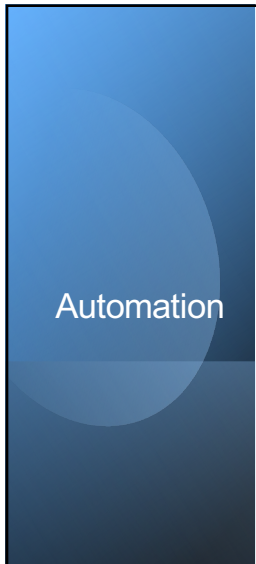
- **Revenue**
 - Contribute directly to the solution of a problem
- **Excise**
 - Do not contribute directly to the solution

23

Examples

- **Compiling a Java class**
 - **Excise task** – necessary but not helping the problem solution
- **Determining methods** that are needed to define a data structure
 - **Revenue task** – necessary and directly helps to solve the problem

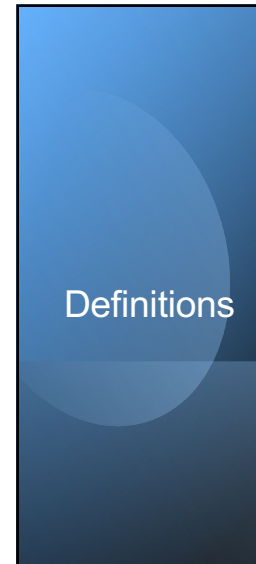
24

A blue vertical bar on the left side of the slide, featuring a large, semi-transparent circle in the center. The word "Automation" is written in white text across the middle of the circle.

Automation

- Excise tasks are natural candidates for automation
- Software testing has more excise task than most parts of the software process
 - Maintaining test scripts, re-running tests, comparing expected results to oracle all consume a lot of engineer time. If we can automate we benefit a lot

25

A blue vertical bar on the left side of the slide, featuring a large, semi-transparent circle in the center. The word "Definitions" is written in white text across the middle of the circle.

Definitions

- Testability
 - The degree to which a system or component facilitates the establishment of test criteria and the performance of tests to determine if these criteria have been met

Determined by two common practical issues... observability and controllability

26

A blue vertical bar on the left side of the slide, featuring a large, semi-transparent circle in the center. The word "Definitions" is written in white text across the middle of the circle.

Definitions

- Software Observability
 - How easy it is to observe the behavior of a program in terms of its outputs, effects on the environment, and other hardware and software components.

27

A blue vertical bar on the left side of the slide, featuring a large, semi-transparent circle in the center. The word "Definitions" is written in white text across the middle of the circle.

Definitions

- Software Controllability
 - How easy it is to provide a program with the needed inputs in terms of values, operations and behaviors

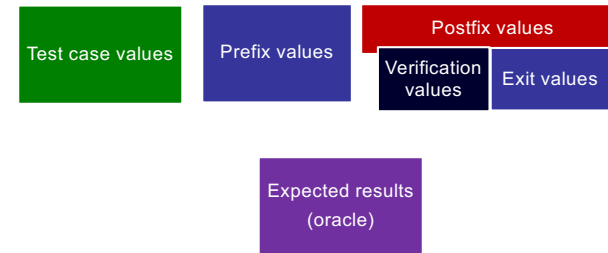
28

Components of a Test Case

- **Test case values**
 - the inputs used to complete an execution of the test
 - These can be simple values, sequences, a web page, etc.

29

Components of a Test Case



30

Help with Controllability and Observability

- **Prefix Values**
 - Inputs necessary to put the software into an appropriate state to receive the test case values
- **Postfix Values**
 - Inputs that need to be sent after the test runs

Example: GUI reaching events

31

Help with Controllability and Observability

- **Verification Values**
 - Necessary to see the results of the test case
- **Exit values**
 - Values or commands needed to terminate the program or return it to a stable state

Evosuite: scaffolding file
handles
startup/teardown

32

Test Oracle

Expected Results

- Decides whether or not a test has passed

33

Test Case Definition

Composed of the *test case values, prefix values, postfix values, and expected results* necessary for completing execution and evaluation of the software under test (sometimes called the SUT)

34

RIPR model Revisited

- Tests are designed to look for a fault in a particular location. The **prefix values** provide **reachability (R)**, the **test case values** achieve infection (**I**), the **postfix values** achieve **propagation (P)** and the **expected results** reveal **failures (R)**.

35

Characteristics of Test Automation Frameworks

- **Assertions** to evaluate expected results
- Ability to **share common test data** between tests
- Test sets to **easily organize and run tests**
- Ability to **run tests from either command line or GUI**

36

Test Automation Framework

- A set of **assumptions, concepts and tools that** support test automation
- Should **include test driver** (runs a test set by executing the software on each test). Should also compare results
- Simplest test driver- main() method of a class
 - Creates instances, manipulate values, etc.

37

The JUnit Test Framework

- **JUnit test scripts**
 - Run as standalone Java programs (command line or from within an IDE)
- Can test an entire class, a method, interacting methods, or interactions between objects (integration)
- Tests put into **Test Methods**
- Test methods put into a **Test Class**

38

Two Parts to JUnit

1. A **collection of test methods**
2. Methods to set up the program state (**prefix values**) and update after the test (**postfix values**)

39

JUnit

- Tests written from methods in the:
 - junit.framework.Assert class
- Each test checks a condition (assertion) and reports pass/fail to the test runner

40

Common Methods

- `assertTrue` (boolean)
simplest form (returns T/F)
- `assertTrue` (String, boolean)
provides information to tester **if the assertion is false** (failure)
- `fail`(String)
used specify a failing message when a specific line is reached

41

Test Fixture

- **State of the test** defined by the current values of key variables in the software
- Can be **shared** between tests
- Objects used in fixtures are declared as instance variables and initialized in a **@Before** method and reset or deallocated in an **@After** method

42

Example

```
Public class Calc{  
    static public int add (int a, int b)  
    {  
        return a+b  
    }  
}  
-----  
import org.junit*;  
import static org.junit.Assert.*;  
public class CalcTest  
{  
    @Test public void testAdd()  
    {  
        assertEquals("Calc sum incorrect", 5==Calc.add(2,3));  
    }  
}
```

Expected output

Test values

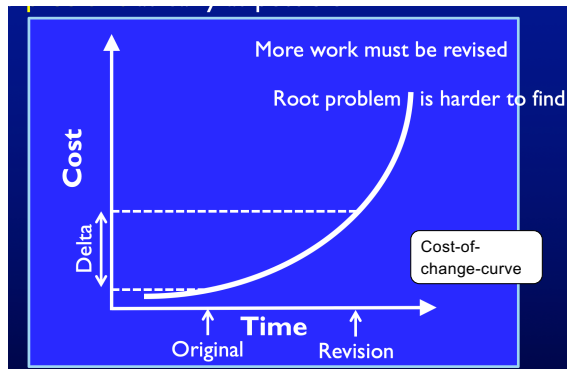
Printed if assert fails

43

Test Driven Development

44

The Increased Emphasis on Testing



Introduction to Software Testing, Edition 2 (Ch 4)

45

Traditional Assumptions

- Modeling and analysis techniques can be effective in identifying problems and changes early in the software lifecycle
- The savings shown in the cost-of-change cure justify the cost of modeling and analysis techniques

46

In Practice

- Requirements are changing
- Customers don't always know up front exactly what they want
- We have moved more to "agile" and iterative methods of development

47

Agile methods

- Dissatisfaction with the overheads involved in design methods led to the creation of agile methods. These methods:
 - Focus on the code rather than the design;
 - Are based on an iterative approach to software development;
 - Are intended to deliver working software quickly and evolve this quickly to meet changing requirements.
- Agile methods are probably best suited to small/medium-sized business systems or PC products.

48

48

Principles of agile methods

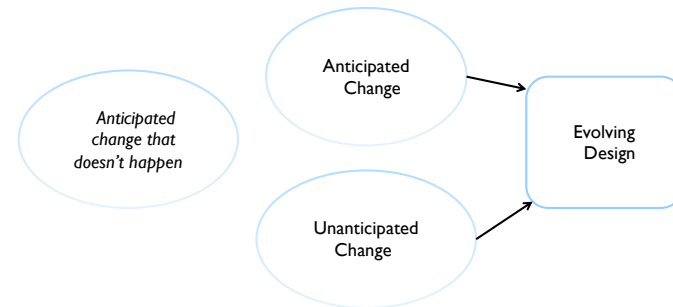
Principle	Description
Customer involvement	The customer should be closely involved throughout the development process. Their role is provide and prioritise new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognised and exploited. The team should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and design the system so that it can accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process used. Wherever possible, actively work to eliminate complexity from the system.

49

49

Supporting Evolutionary Design

Traditional design advice says to anticipate changes
Designers often anticipate changes that don't happen



Both anticipated and unanticipated changes affect design

Introduction to Software Testing, Edition 2 (Ch 4)

© Ammann & Offutt

50

A Limited View of Correctness

- In traditional methods, we try to define all correct behavior completely, at the beginning
 - What is correctness?
 - Does "correctness" mean anything in large engineering products?
 - People are VERY BAD at completely defining correctness

Introduction to Software Testing,
Edition 2 (Ch 4)

© Ammann & Offutt

51

A Limited View of Correctness

- In agile methods, we redefine correctness to be relative to a specific set of tests
 - If the software behaves correctly on the tests, it is "correct"
 - Instead of defining all behaviors, we demonstrate some behaviors
 - Mathematicians may be disappointed at the lack of completeness

Introduction to Software Testing,
Edition 2 (Ch 4)

© Ammann & Offutt

52

Continuous Integration

- Agile methods work best when the current version of the software can be run against all tests at any time

A continuous integration server rebuilds the system, returns, and re-verifies tests whenever any update is checked into the repository

- Mistakes are caught earlier
- Other developers are aware of changes early
- The rebuild and re-verify must happen as soon as possible

A continuous integration server doesn't just run tests, it decides if a modified system is still correct

Introduction to Software Testing, Edition 2 (Ch 4)

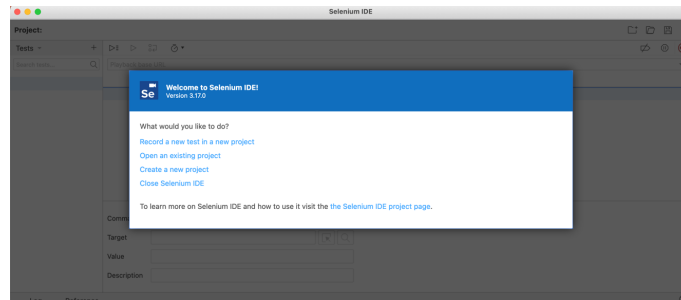
© Ammann & Offutt

53

Automated Testing Tools

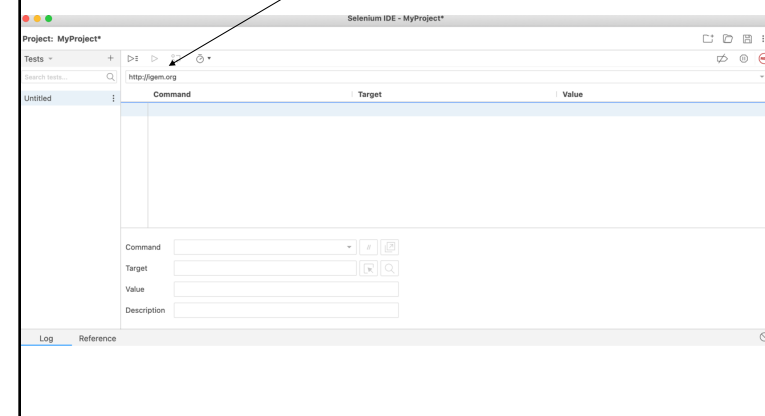
54

Starting Selenium



55

Base URL



56

Evosuite



<http://evosuite.org>

57

Demo

```
[mcohen@pyrite-n3 evosuite]$ ls
evosuite-1.1.0.jar  evosuite-standalone-runtime-1.1.0.jar  lib  Tutorial_Stack
evosuite-report  evosuite-tests  Triangle
[mcohen@pyrite-n3 evosuite]$ javac Tutorial_Stack/tutorial/*.java
Note: Tutorial_Stack/tutorial/Stack.java uses unchecked or unsafe operations.
[mcohen@pyrite-n3 evosuite]$ java -jar evosuite-1.1.0.jar -projectCP Triangle -class triangle.TriangleType
* EvoSuite 1.1.0
* Going to generate test cases for class: triangle.TriangleType
* Starting Client-0
* Connecting to master process on port 17649
* Analyzing classpath:
  - Triangle
* Finished analyzing classpath
* Generating tests for class triangle.TriangleType
* Test criteria:
  - Line Coverage
  - Branch Coverage
  - Exception
  - Mutation testing (weak)
  - Method-Output Coverage
  - Top-Level Method Coverage
  - No-Exception Top-Level Method Coverage
  - Context Branch Coverage
[Progress: > 0%] [Cov: > 0%] * Total number of test g
oals for DYNAMOSA: 173
* Using seed 1613833653736
* Starting evolution
* Initial Number of Goals in DynaMOSA = 13 / 173
[Progress: > 3%] [Cov: =====> 92%]
```

58

The Tests

```
public class Stack_ESTest extends Stack_ESTest_scaffol

@Test(timeout = 4000)
public void test0() throws Throwable {
    Stack<Object> stack0 = new Stack<Object>();
    assertTrue(stack0.isEmpty());

    Integer integer0 = new Integer((-20));
    stack0.push(integer0);
    stack0.push(integer0);
    stack0.pop();
    assertFalse(stack0.isEmpty());
}

@Test(timeout = 4000)
```

59

Fuzzing A Triangle Program

```
#include <string.h>
int main()
{
    int side1, side2, side3;
    char name[12];
    /* Input sides of a triangle */
    // printf("Enter three sides of triangle: ");
    scanf("%d%d%d", &side1, &side2, &side3);
    if(side1==side2 && side2==side3)
    {
        /* If all sides are equal */
        strcpy(name, "Equilateral");
    }
    else if(side1==side2 || side1==side3 || side2==side3)
    {
        /* If any two sides are equal */
        strcpy(name, "Isosceles");
    }
    else
    {
        /* If none sides are equal */
        char result[12] = "This is a scalene triangle";
        strcpy(name, result);
    }
    printf("%s\n", name);
    return 0;
}
```

60

Fuzzing A Triangle Program

american fuzzy lop 2.57b (triangle)

process timing	overall results
run time : 0 days, 0 hrs, 0 min, 7 sec	cycles done : 0
last new path : 0 days, 0 hrs, 0 min, 6 sec	total paths : 4
last uniq crash : 0 days, 0 hrs, 0 min, 7 sec	uniq crashes : 1
last uniq hang : none seen yet	uniq hangs : 0
cycle progress	map coverage
now processing : 0 (0.00%)	map density : 0.00% / 0.02%
paths timed out : 0 (0.00%)	count coverage : 1.00 bits/tuple
stage progress	findings in depth
now trying : arith 8/8	favored paths : 2 (50.00%)
stage execs : 89/465 (19.14%)	new edges on : 4 (100.00%)
total execs : 385	total crashes : 1 (1 unique)
exec speed : 27.12/sec (slow!)	total tmouts : 151 (1 unique)
fuzzing strategy yields	path geometry
bit flips : 3/56, 0/55, 0/53	levels : 2
byte flips : 0/7, 0/6, 0/4	pending : 4
arithmetics : 0/0, 0/0, 0/0	pend fav : 2
known ints : 0/0, 0/0, 0/0	own finds : 2
dictionary : 0/0, 0/0, 0/0	imported : n/a
havoc : 0/0, 0/0	stability : 100.00%
trim : 0.00%/1, 0.00%	

[cpu000: 46%]

61

Automated GUI Testing

62

Modeling Event Interactions

- **Event flow model goal:**
 - Represent all possible event interactions on the GUI
 - This will allow a model that can be traversed for test case generation
- **Graph model:**
 - Vertex represents an event (click-on-edit, click-on-paste)
 - An edge from vertex x to vertex y shows that an event y can be performed immediately after event x
 - y follows x

63

Graph model

- Analogous to CFG (control flow graph)
- A state machine model equivalent to this graph can be constructed; state would represent possible events that can be executed on the GUI, transitions cause state changes
- In general a GUI that has no restrictions on event ordering and no windows/menus would have a fully connected graph; in practice this is not the case

64

More on the Graph Model

- Modal windows:
 - A GUI window that once invoked monopolizes the GUI interaction with user/program
 - Restricts focus of the user to a specific range of events within the window until terminated
- **Modeless windows:**
 - Do not restrict events; instead they expand the possible set of events available to the user

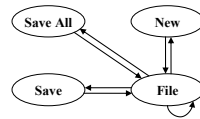
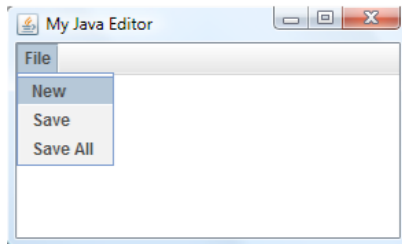
65

Event Flow Graph

- Event interactions within a modal dialogue can be represented as an **event-flow graph** (EFG)
- An EFG represents all possible event sequences that can be executed in a dialogue
- Once all of the modal dialogues of a GUI have been modeled as EFGs then the interactions between EFGs can be identified and modeled
- **Integration tree:** identifies interactions (invocations) between modal dialogs

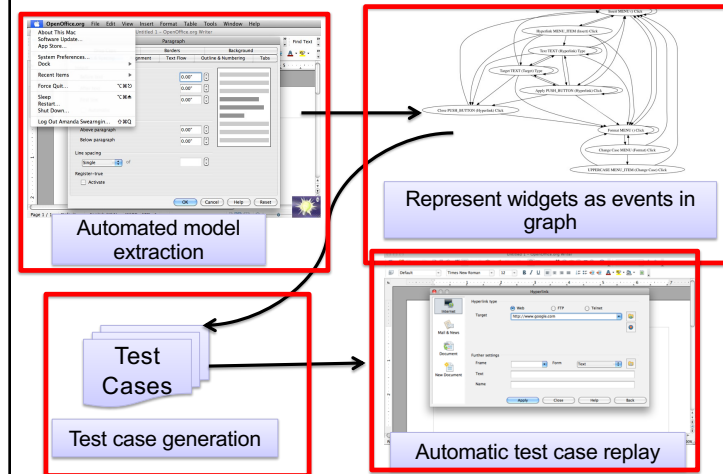
66

Example EFG



67

Graph-Based Approach

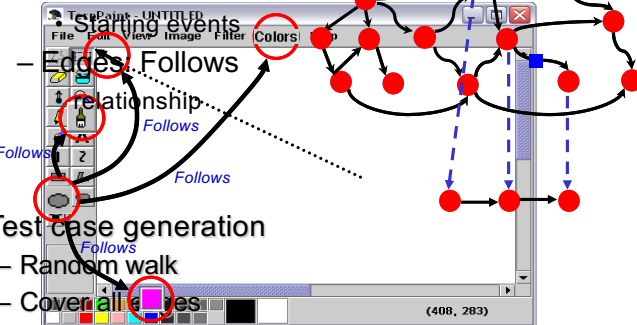


68

Sampling

Modeling The Event-Interaction Space

- Event flow graph (EFG)
 - Nodes: all GUI events



- Test case generation
 - Random walk
 - Cover all edges

69

GUITAR Testing Tool

1. Rip interface (creates gui file)
2. Build efg from gui file
3. Generate test cases from efg – (length 2, length 3, random)
4. Run test cases

70

COMS 4170 Software Testing

Chapter 5 – Coverage

71

Example : Jelly Bean Coverage

Flavors :

1. Lemon
2. Pistachio
3. Cantaloupe
4. Pear
5. Tangerine
6. Apricot



Colors :

1. Yellow (Lemon, Apricot)
2. Green (Pistachio)
3. Orange (Cantaloupe, Tangerine)
4. White (Pear)

■ Possible coverage criteria :

1. Taste one jelly bean of **each flavor**
 - Deciding if **yellow jelly bean is Lemon or Apricot** is a **controllability** problem
2. Taste one jelly bean of **each color**

Introduction to Software Testing, Edition 2 (Ch 5)

© Armann & Offutt

72

72

Coverage

Given a set of test requirements TR for coverage criterion C , a test set T satisfies C coverage if and only if for every test requirement tr in TR , there is at least one test t in T such that t satisfies tr

- **Infeasible test requirements**: *test requirements that cannot be satisfied*
 - No test case values exist that meet the test requirements
 - Example: Dead code
 - Detection of infeasible test requirements is formally undecidable for most test criteria
- Thus, 100% coverage is **impossible** in practice

Introduction to Software Testing, Edition 2 (Ch 5)

© Armin & Offutt

73

73

More Jelly Beans

$T1 = \{ \text{three Lemons, one Pistachio, two Cantaloupes, one Pear, one Tangerine, four Apricots} \}$

- Does test set T1 satisfy the flavor criterion ?

$T2 = \{ \text{One Lemon, two Pistachios, one Pear, three Tangerines} \}$

- Does test set T2 satisfy the flavor criterion?
- Does test set T2 satisfy the color criterion?

Introduction to Software Testing, Edition 2 (Ch 5)

© Armin & Offutt

74

74

Definition

- **Coverage Criterion**:
 - A rule or collection of rules that impose test requirements on a test set
- **Coverage**:
 - Given a set of test requirements, TR , for a coverage criterion C , a test set satisfies C , if and only if, for every test requirement, tr , in TR , at least one test, t , in TR exists that satisfies tr

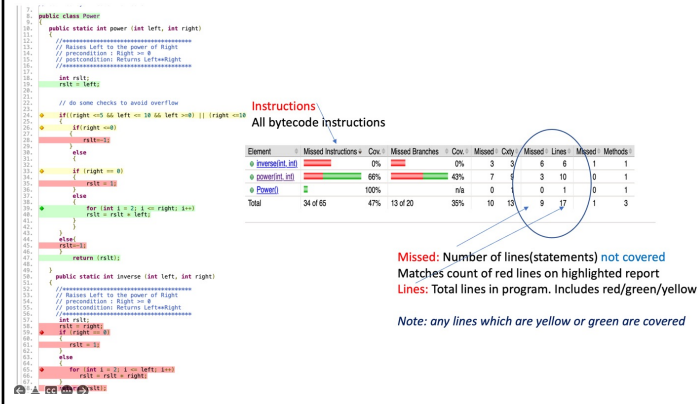
75

Examples from Class

- **TR**
 - Cover all statements in a program
- **Approach**
 - Use Cobertura. Keep adding tests until all statements covered
 - What if we can't cover all statements?
 - We can measure the percentage covered – this gives us a quality metric

76

Statement Coverage with Jacoco



77

Definition

- **Minimal Test Set**
- Given TR and a test set T that satisfies all test requirements, T is **minimal** if removing any single test from T will cause T to no longer satisfy all test requirements

78

Definition

- **Minimum Test Set**
- Given TR and a test set T that satisfies all test requirements, T is minimum if there is **no smaller set of test that also satisfy all test requirements**

79

Definition

- **Coverage Level**
 - Give a set of requirements TR and a test set T , the coverage level is the ratio of the number of test requirements satisfied by T to the size of TR .

JaCoCo Maven plug-in example with Offline Instrumentation > default

default

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
TriangleType	93%	81%	5	13	1	10	1	2	0	1	0	1
Triangle	100%	n/a	0	1	0	2	0	1	0	1	0	1
Total	3 of 93	96%	4 of 22	81%	5	14	1	12	1	3	0	2

80

Completeness

- Some requirements cannot be satisfied

```
...
...
x=y+1;
if(x <=y){
    System.out.println("not reachable");
}
```

81

Infeasible Test Requirements

- No test case value exists that meets an *infeasible test requirement*
- Example:
 - Dead code
- Detection of infeasible test requirements is formally undecidable for most coverage
 - therefore 100% coverage is usually impossible in practice

82

Using Coverage Criterion

1. **Generator:** Used to generate test case values to satisfy this criterion. May be hard without good automated tools
(we did this with Evosuite)
2. **Recognizer:** Create tests and then measure coverage (by running tests)
(we did this with Jacoco)

83

Subsumption

- We can often compare coverage criteria to each other
- Jelly Beans**
- **Flavor criterion**
 - Requires each flavor tested once
- **Color criterion**
 - Try each color once
- *Flavor subsumes color!*

84